# LeakLens

## Water Leak Localization System

The Complete Project Report
in Plain Language

What it is, how it works, how we test it, and what every number means

**B.Tech Project, Semester 1**

Indian Institute of Information Technology Kottayam

**Guide:** Dr. Divya Sindhu Lekha

Ashish Ranjan (2023BCS0027) • Nidamanuri Kedar Vaishnav (2023BCS0163)
Abhinandh Rajeev (2023BCS0074) • Kadthala Hrishith Reddy (2023BCS0168)

October 2026

Every result in this report comes from computer simulation of water networks (EPANET through the WNTR tool). No real sensors or real utility data were used. The numbers show how the method behaves on three standard test networks; they are not a promise about performance in a real city.

# Contents


**1 The project at a glance** **3**

**2 Why this matters** **3**

**3 The idea in plain words** **3**

**4 The water networks** **4**

**5 The scenarios** **4**

5.1 What each event does to the network . . . . . 4

5.2 Sensor noise . . . . . 5

**6 How the system analyses one scenario** **5**

6.1 A worked example . . . . . 6

**7 How the system is trained and tested** **6**

**8 What the metrics mean** **7**

8.1 Detection: is something wrong? . . . . . 7

8.2 Classification: what kind of event? . . . . . 8

8.3 Localization: where is the leak? . . . . . 8

**9 Results** **8**

9.1 Detection . . . . . 8

9.2 Which kind of event? . . . . . 9

9.3 Where is the leak? . . . . . 10

9.4 By type of leak . . . . . 10

9.5 Leak size . . . . . 11

9.6 A network the system has never seen . . . . . 11

9.7 Training versus test . . . . . 11

**10 The software** **12**

10.1 Backend . . . . . 12

10.2 Dashboard . . . . . 12

10.3 What the screen says, and where it comes from . . . . . 14

**11 How to run it** **14**

**12 Honest limitations** **14**

**13 What comes next (Semester 2)** **15**

14 Glossary 15

# 1 The project at a glance

**What problem does it solve?** A city's water pipes leak. Finding *that* something is wrong is not enough: a pump stopping, a hydrant opening or a tank running low also change the water pressure. LeakLens tells these apart and, when it is a leak, points to *where* it is.
**What does it do?** It simulates water networks on a computer, compares each situation with a normal day, follows the water to see whether any is disappearing, decides what kind of event it is, and ranks the places to look.
**How is it delivered?** A web dashboard (LeakLens) backed by a small server, so that a person can pick a situation, press one button and read a plain-language verdict.

**Headline results** (measured only on 139 test scenarios that the system never saw while learning):

| Question | Answer |
|---|---|
| Is the leak's exact node the first one the system lists? | **96.7%** of 60 test leaks |
| Is the real leak among the first five nodes listed? | **98.3%** |
| How often does a normal day trigger a false alarm? | **0.0%** (a fixed 0.5 m alarm: 41.7%) |
| How often is the *kind* of event named correctly? | **82.7%** (on scenarios it learned from: 95.6%) |
| Time to analyse one scenario | about 22 milliseconds |

The sections that follow explain every one of these, how it is measured, and where the system is weak.

# 2 Why this matters

Water utilities lose a large share of the treated water they produce to leaks and unmetered use, often called non-revenue water. The literature reports leakage from about 3% to over 50% depending on maintenance [1], and our presentation quotes about 40% for developing countries. Leaks waste water and energy, damage roads and buildings and can let dirt into the pipes.

Finding leaks by walking the streets with listening equipment is slow. A pressure or flow sensor can show that something changed, but a pressure drop alone is ambiguous: a hot day, a pump failure or a leak can all lower the pressure. The practical question is therefore:

> *When pressure and flow change in a water network, is it a leak or just a normal change in operation, and if it is a leak, which node is it?*

Related research tests one network at a time, treats "is there a leak?" and "where?" as separate problems, or uses black-box models that are hard to explain [2–5]. Our aim is an explainable pipeline that is tested on several networks and says *why* it reached its answer.

# 3 The idea in plain words

Think of a bank account. Money comes in, customers spend some, and the rest stays. If money goes missing and no customer spent it, someone took it. A water node works the same way:

**water in − water out = water customers use + water that leaks**

If we know how much water the customers at each node really use (from smart meters), then any water that leaves a node and is not accounted for by a customer meter must be leaking. We call this **unaccounted flow**.

A leak makes a large unaccounted flow at one node. A demand change does not, because the meters record it. A pump stopping or a tank changing alters the pressure across a wide area but loses no water.

That one idea, plus careful handling of sensor noise, does most of the work in LeakLens.

## 4 The water networks

We use three standard benchmark networks supplied with EPANET, the engineering standard for modelling water pipe networks. Each is a graph: junctions (where customers draw water) joined by pipes, with reservoirs (sources), tanks (storage) and pumps.

| Network | Junctions | Pumps | Tanks | Reservoirs |
|---|---|---|---|---|
| Net1 (small) | 9 | 1 | 1 | 1 |
| Net2 (medium) | 35 | 0 | 1 | 0 |
| Net3 (large benchmark) | 92 | 2 | 3 | 2 |

Net2 has no reservoir or pump; one junction with negative demand acts as its supply.

## 5 The scenarios

A **scenario** is one 24-hour computer simulation of a network in which one thing (or a few things together) differs from the normal day. We built 617 scenarios on the three networks. For each we save, every hour, the pressure at every node, the flow in every pipe and the water actually delivered to customers at every node, together with the *true answer* (what was changed and where) so that we can later check the system.

| **Kind** | Count | What is changed |
|---|---|---|
| **Leak** | 299 | A hole opens at one junction. Five sizes (tiny to burst), every junction, four start times, with and without sensor noise, and hidden behind another event (hydrant, demand surge, blocked pipe or pump outage). |
| **Demand change** | 60 | One node uses 1.5× or 0.5× as much; all nodes use 1.3× or 0.7×; or a hydrant is opened for four hours. |
| **Pump or reservoir** | 54 | A pump is switched off for 2, 4 or 6 hours; or a reservoir level is raised or lowered by 2, 5 or 10 metres. |
| **Tank change** | 60 | A tank starts 10–90% full, or is cut off from the network. |
| **Blocked pipe** | 48 | A busy pipe is closed, or is partly blocked. |
| **Pipe ageing** | 42 | All pipes become rougher (older, scaled pipes waste more pressure). |
| **Normal day** | 54 | Nothing changes; only random sensor errors, small or large, or none. |

### 5.1 What each event does to the network

- **Leak.** Water escapes through the hole. The amount grows with pressure: $Q = 0.75\,A\sqrt{2gP}$ for a hole of area $A$ at pressure head $P$. Flow towards the leak rises and pressure falls around it, most at the leak. The loss is *local* and *unmetered*.
- **Demand change.** Customers use more or less. Pressure falls where the use is heavy, much like a leak, but the extra flow is *metered*.
- **Pump or reservoir.** The places where water and pressure enter the network change. A pump switched off removes the energy it adds, so pressure falls over a wide area; a reservoir shifted by 5 m shifts the pressure of the whole network by about 5 m. No water is lost.
- **Tank change.** A tank that starts low or is cut off changes how much it supplies; flow reroutes. No water is lost.

- **Blocked pipe.** The pipe's flow is forced elsewhere. Pressure falls downstream and may rise upstream. No water is lost.
- **Pipe ageing.** Rougher pipes lose more pressure for the same flow, slowly and widely. No water is lost.

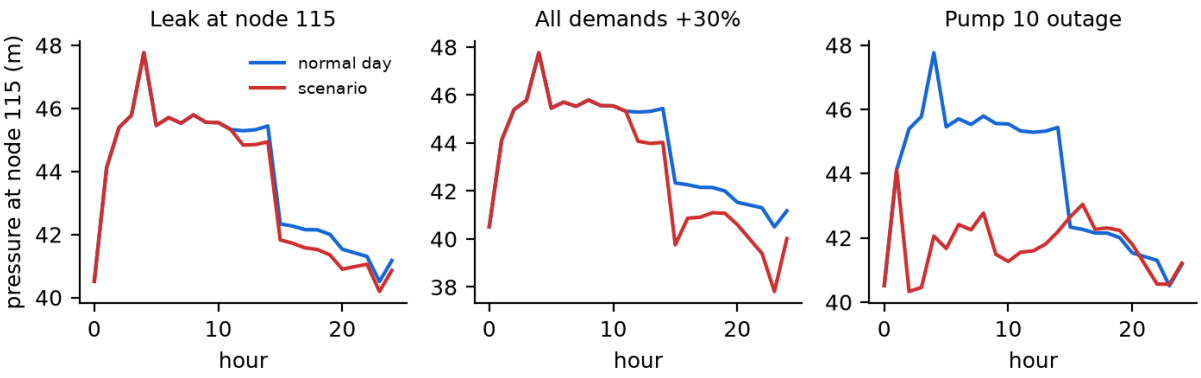
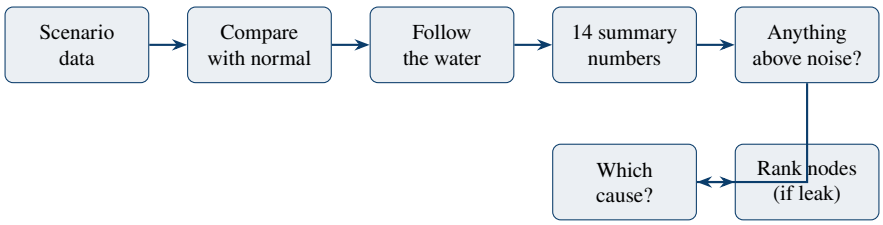



**Figure 1:** Pressure at one Net3 node on a normal day (blue) and in three scenarios (red): a leak (steps down from noon), a demand surge (lower all day) and a pump outage (drops while the pump is off).

### 5.2 Sensor noise

Real sensors are not perfect. In "noisy" scenarios we add random errors (pressure ±0.05 or 0.2 m; flows and demands 0.5% or 2%). The normal-day reference is kept clean: it stands for what the hydraulic model expects.

## 6 How the system analyses one scenario




**Step 1: compare with the normal day.** For every node and hour we subtract the scenario's pressure from the normal day's. A drop only counts if it stays above the noise threshold for two hours in a row, so one odd reading cannot trigger anything.

**Step 2: learn what noise looks like.** Each network has 18 normal runs that contain only sensor noise; the learning (training) ones are used to measure how big the noise normally is. The pressure threshold is four times that spread, about 0.5–0.6 m. A fixed 0.5 m alarm ignores this and raises many false alarms (41.7% of normal test days).

**Step 3: follow the water.** For each node we work out how much water went in minus out, subtract the change in customer use, and what remains is the *unaccounted flow*, which is the leak flow.

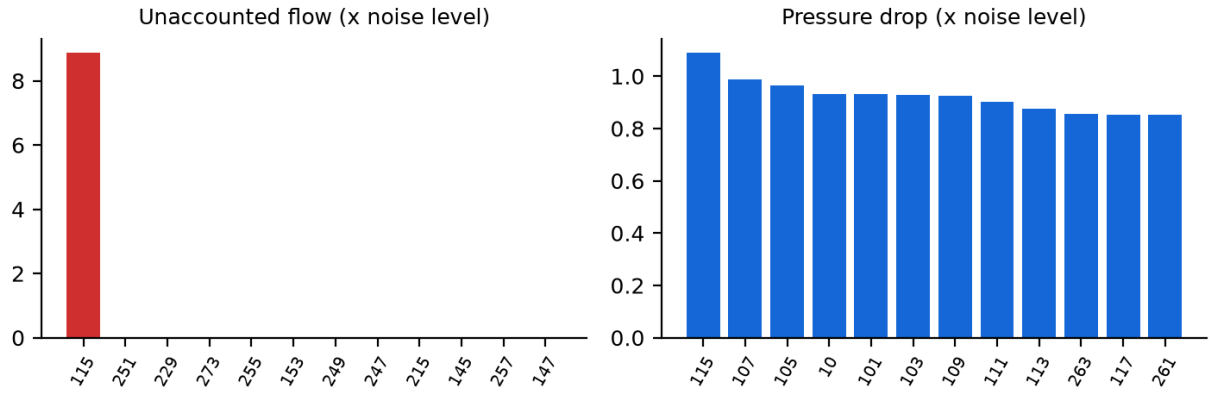



**Figure 2:** A leak at one Net3 node. Left: unaccounted flow is large at the leak node and nearly zero elsewhere. Right: the pressure drop is high at the leak but also at its neighbours, so pressure alone is blurrier.

**Step 4: fourteen summary numbers.** Each scenario is boiled down to 14 numbers, for example: the share of junctions that lost pressure, the biggest unaccounted flow, how much flow a pump lost, how much the reservoir supply and the tanks changed. All are measured in units of the noise level or as a share of total demand, so they are comparable between networks.

**Step 5: is anything above noise?** If every number is within noise, the answer is *normal day* and the analysis stops. This is why normal days do not cause false alarms. It also means leaks smaller than the noise cannot be seen.

**Step 6: which cause?** A *random forest* (300 small decision trees that vote) looks at the 14 numbers and gives a probability for each of: normal, leak, demand change, pump/reservoir, tank, blocked pipe, ageing. The highest probability is the verdict; that probability is the confidence (High from 85%, Medium from 60%, otherwise Low). This is the only step that is *learned* from data; everything else is calculation.

**Step 7: rank the nodes (leaks only).** Each junction gets a score

$$\text{score} = A + 0.3\,D,$$

where $A$ is its unaccounted flow and $D$ its pressure drop, each scaled so that the largest in the network is 1. The unaccounted flow is the main signal; the pressure drop is a backup. Nodes are sorted by score: the first is the system's best guess, the next ones are where to look if it is wrong. The ranking is a fixed formula, not a trained model.

### 6.1 A worked example

In Net3 a leak is opened at node 115. The leak node has the largest unaccounted flow ($A = 1.00$) and the largest pressure drop ($D = 1.00$), so its score is $1.00 + 0.3 \times 1.00 = 1.30$. Its neighbours, nodes 107 and 105, have almost no unaccounted flow ($A \approx 0$) but a large pressure drop ($D \approx 0.9$), so they score only about 0.27. Node 115 is clearly first. This is a "Top-1 success": the true leak is the first node on the list, zero pipes away.

## 7 How the system is trained and tested

> If you test a system on the same examples it learned from, it can score perfectly just by remembering them, and you learn nothing about new situations. So we keep some scenarios completely aside.

All 617 scenarios were split once, with a fixed random seed, into about **80% for learning** and **20% for testing**.

| Kind | All | Learning (train) | Testing (test) |
|---|---|---|---|
| Normal day | 54 | 42 | 12 |
| Leak | 299 | 232 | 67 |
| Demand change | 60 | 48 | 12 |
| Pump / reservoir | 54 | 42 | 12 |
| Tank change | 60 | 48 | 12 |
| Blocked pipe | 48 | 36 | 12 |
| Pipe ageing | 42 | 30 | 12 |
| Total | 617 | 478 | 139 |

- **Learned only from the training scenarios:** the noise thresholds and the random forest.
- **Measured only on the test scenarios:** every accuracy, false-alarm and Top-$k$ figure in this report. The system never saw these while learning.
- **Near-copies stay together.** All leaks at the same junction (every size, start time, noise level and "hidden behind" variant) go to the same side, and a scenario goes with its sensor-noise twin. Otherwise a test scenario could be almost identical to a training one, and the test would be too easy.
- **Fair mix.** Each network and each kind of event is split about 80/20.
- The dashboard tags every scenario *Unseen test* or *Training*, and shows unseen ones by default.

The leak-ranking formula (weight 0.3, no neighbour smoothing) and the cut-offs in step 5 were chosen during development while looking at all the scenarios, before the split existed. They were not re-tuned afterwards. The test set is small, so each percentage has a wide margin of error.

# 8 What the metrics mean

This section explains every number used later, in plain words, with a small example. Think of the system's job in three parts: noticing a problem (detection), naming it (classification) and finding it (localization).

## 8.1 Detection: is something wrong?

Here a scenario is "positive" if any abnormal event happened (leak, demand change, pump, tank, blockage, ageing) and "negative" if it is a normal day. Comparing the system's call with the truth gives four counts:

| | System says problem | System says normal |
|---|---|---|
| Truth: problem | true positive (TP) | false negative (FN, a miss) |
| Truth: normal day | false positive (FP, false alarm) | true negative (TN) |

- **Precision** = $\frac{TP}{TP+FP}$. When the system raises an alarm, how often is it real? 100% means no false alarms.
- **Recall** = $\frac{TP}{TP+FN}$. Of the real problems, how many did it catch? 100% means none missed.
- **F1 score** = $\frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision}+\text{recall}}$. One number that is high only when both precision and recall are high.
- **False-alarm rate** = $\frac{FP}{FP+TN}$. Of the normal days, how many wrongly raise an alarm?

*Example from our test.* For the full system: $TP = 119$, $FP = 0$, $FN = 8$, $TN = 12$. So precision $= 119/(119+0) = 100.0\%$, recall $= 119/(119+8) = 93.7\%$, F1 $= 96.7\%$, false-alarm rate $= 0/(0+12) = 0.0\%$ (on 12 normal test days).

### 8.2 Classification: what kind of event?

- **Accuracy** = correct answers / all test scenarios. Easy to read but it can hide weak spots in small classes.
- **Per-class precision, recall and F1.** The same three ideas as above, worked out separately for each kind of event. For "leak": precision is how often a "leak" verdict is really a leak; recall is how many real leaks get a "leak" verdict.
- **Macro-F1** = the plain average of the per-class F1 scores. Every kind counts equally, so a weak small class pulls it down.
- **Confusion matrix.** A table: each row is the true kind, each column what the system said. A perfect system has numbers only on the diagonal.
- **Train accuracy versus test accuracy.** Accuracy on the scenarios it learned from versus on unseen ones. A big gap means the model partly memorises.
- **Leave-one-network-out.** Train on two networks, then test on the third network's test scenarios. It shows whether the system works on a network it has never seen.

### 8.3 Localization: where is the leak?

- **Rank.** Position of the true leak node in the system's sorted list (1 means first).
- **Top-1 accuracy** = $\frac{1}{N}\sum \mathbf{1}[\text{rank} = 1]$: the share of leaks whose exact node is first. **Top-3** and **Top-5**: the true node is within the first 3 or 5.
- **Pipe hops (error).** The number of pipes between the system's first guess and the true node. 0 is exact; 1 is the next junction along the pipe. We report the average.
- **Within 1 hop.** Share of leaks where the first guess is the exact node or its direct neighbour.
- **Error in metres.** The length of pipe on the shortest path between guess and truth.
- **End-to-end.** Counts a leak as a success only if the system both called it a leak *and* put the right node first.
- **Inference time.** How long one analysis takes.

*Example.* Over 60 test leaks, the exact node came first 58 times: Top-1 = 58/60 = 96.7%. The first version managed 35 of 60 (58.3%). With only 60 leaks, one miss changes the percentage by about 1.7 points.

## 9 Results

### 9.1 Detection

| Method | Precision | Recall | F1 | False alarms |
|---|---|---|---|---|
| Fixed 0.5 m alarm (first version) | 95.7% | 87.4% | 91.4% | 41.7% |
| Learned pressure threshold | 100.0% | 77.2% | 87.1% | 0.0% |
| Full system (detector + cause model) | 100.0% | 93.7% | 96.7% | 0.0% |

The fixed 0.5 m alarm catches most problems but raises a false alarm on 41.7% of normal days, because it ignores sensor noise. A threshold learned from each network's own normal runs, plus the "anything above noise?" check, raises none on the test days, while still catching 93.7% of real problems.

## 9.2 Which kind of event?

Accuracy on unseen test scenarios: **82.7%** (on the training scenarios: 95.6%). Macro-F1: 76.4%. Of the leaks, 89.6% were named as leaks. All seven that were not are leaks at Net3 node 601, a pump-outlet node we exclude as unreliable (its pressure swings wildly for tiny changes); every other test leak was named a leak. Only 1.4% of non-leaks were wrongly called a leak.

| Kind | Precision | Recall | F1 | Test scenarios |
|---|---|---|---|---|
| Normal day | 60.0% | 100.0% | 75.0% | 12 |
| Leak | 98.4% | 89.6% | 93.8% | 67 |
| Demand change | 100.0% | 100.0% | 100.0% | 12 |
| Pump / reservoir | 66.7% | 66.7% | 66.7% | 12 |
| Tank change | 57.1% | 66.7% | 61.5% | 12 |
| Blocked pipe | 100.0% | 75.0% | 85.7% | 12 |
| Pipe ageing | 54.5% | 50.0% | 52.2% | 12 |

Confusion matrix on the test scenarios (rows = truth, columns = system's answer; a dot means zero):

| | Normal | Leak | Demand | Pump | Tank | Blocked | Ageing |
|---|---|---|---|---|---|---|---|
| Normal day | 12 | . | . | . | . | . | . |
| Leak | 6 | 60 | . | . | 1 | . | . |
| Demand change | . | . | 12 | . | . | . | . |
| Pump / reservoir | 2 | . | . | 8 | 2 | . | . |
| Tank change | . | . | . | . | 8 | . | 4 |
| Blocked pipe | . | 1 | . | . | 1 | 9 | 1 |
| Pipe ageing | . | . | . | 4 | 2 | . | 6 |

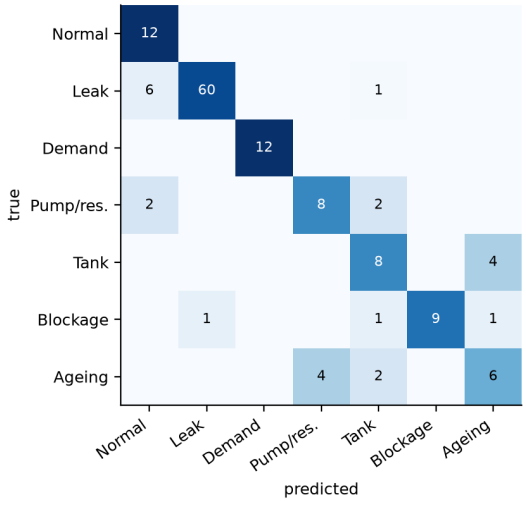



**Figure 3:** The same matrix as a picture.

**Reading it:** leaks and demand changes are recognised reliably. Pump, tank, blockage and ageing are sometimes mistaken for each other because each makes pressure fall across a wide area without losing water; some small events are reported as normal because they stay within the sensor noise.

## 9.3 Where is the leak?

| Method (60 test leaks) | Top-1 | Top-3 | Top-5 | Avg. hops | Within 1 hop |
|---|---|---|---|---|---|
| First version (pressure + 50 × flow) | 58.3% | 70.0% | 71.7% | 3.35 | 58.3% |
| **Final method** | 96.7% | 98.3% | 98.3% | 0.13 | 96.7% |
| Final + neighbour smoothing | 96.7% | 98.3% | 98.3% | 0.13 | 96.7% |
| Final, without demand meters | 93.3% | 98.3% | 98.3% | 0.72 | 93.3% |
| Pressure only | 60.0% | 70.0% | 70.0% | 3.05 | 61.7% |
| Pressure only + smoothing | 50.0% | 63.3% | 65.0% | 3.18 | 58.3% |

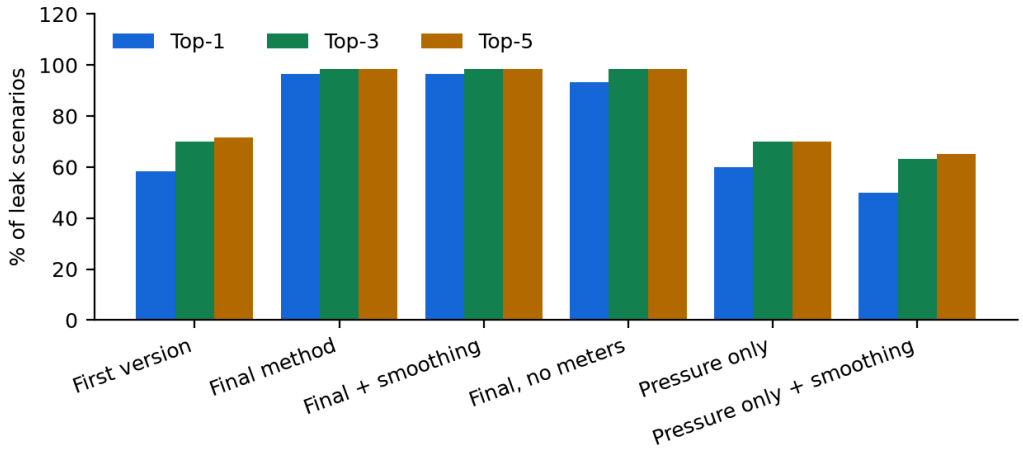



**Figure 4:** Localization accuracy of each method on the test leaks.

- **Unaccounted flow is what works.** Using it lifts Top-1 from 58.3% to 96.7%. The final method is on average 0.13 pipes away (about 60 m) from the true node; 96.7% of leaks are found at the node or its neighbour. End-to-end success is 96.7%.
- **Without demand meters** Top-1 is 93.3%: a demand change at another node can look like a small leak.
- **Pressure alone** reaches only 60.0%, because the pressure drop spreads to neighbouring nodes.
- **Smoothing the scores over neighbours** gave 96.7%, no improvement, so it is not used. Network distance is used instead to measure the error and to define the search region.

## 9.4 By type of leak

| Group (test leaks) | Leaks | Top-1 | Top-5 |
|---|---|---|---|
| Leak hidden behind another event | 10 | 100.0% | 100.0% |
| Leak with sensor noise | 4 | 100.0% | 100.0% |
| Leak size variations | 20 | 90.0% | 95.0% |
| Leak start time variations | 4 | 100.0% | 100.0% |
| Leak at a junction (clean data) | 22 | 100.0% | 100.0% |
| Dead End | 17 | 100.0% | 100.0% |
| Hub | 1 | 100.0% | 100.0% |
| Inline | 42 | 95.2% | 97.6% |
| NET1 | 11 | 100.0% | 100.0% |
| NET2 | 26 | 92.3% | 96.2% |
| NET3 | 23 | 100.0% | 100.0% |

Leaks hidden behind another event are still found, because a hydrant or demand surge is metered and cancels out of the unaccounted flow. Small groups mean small differences should not be over-read.

## 9.5 Leak size

| Hole area ($m^2$) | Test leaks | Noticed by pressure | Called a leak | Exact node (Top-1) |
|---|---|---|---|---|
| 0.0001 | 4 | 75.0% | 100.0% | 100.0% |
| 0.0003 | 4 | 100.0% | 100.0% | 100.0% |
| 0.001 | 4 | 100.0% | 100.0% | 100.0% |
| 0.003 | 4 | 100.0% | 100.0% | 100.0% |
| 0.01 | 4 | 100.0% | 100.0% | 50.0% |

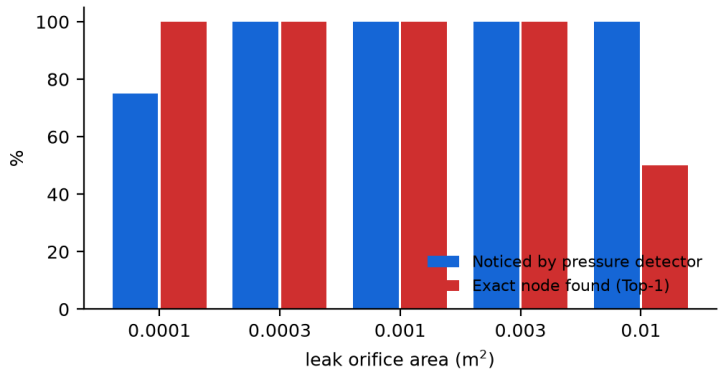



**Figure 5:** Detection and exact-node success by leak size.

Very small leaks sit close to the sensor noise; some are not noticed. The biggest leak lowers pressure across the whole network, so its exact node is the hardest to pick. There are only four test leaks per size, so this is indicative.

## 9.6 A network the system has never seen

Train on the training scenarios of two networks, test on the third network's test scenarios:

| Unseen network | Test scenarios | Overall accuracy | Leak recall | Leak precision | Accuracy without meters |
|---|---|---|---|---|---|
| NET1 | 33 | 48.5% | 100.0% | 100.0% | 45.5% |
| NET2 | 44 | 81.8% | 92.3% | 100.0% | 75.0% |
| NET3 | 62 | 54.8% | 73.3% | 100.0% | 14.5% |

Leaks mostly carry over to a new network. Telling pump, tank, blockage and ageing apart does not carry over well, and without demand meters the result on Net3 collapses to 14.5%. This is the most important thing to improve in Semester 2.

## 9.7 Training versus test

Cause accuracy: **95.6%** on training scenarios, **82.7%** on test scenarios. Leak Top-1: **97.8%** on training leaks, **96.7%** on test leaks. The cause model shows a gap (it partly memorises); the ranking shows almost none, because it is a fixed formula that never trained.

# 10 The software

## 10.1 Backend

A FastAPI server runs the analysis and serves data to the dashboard. At start-up it creates an **SQLite** database and rebuilds a table of all scenarios from the saved files, including the Unseen test / Training tag, so the dashboard always lists exactly what exists.

| Endpoint | What it returns |
|---|---|
| `GET /api/networks` | the three networks and how many scenarios each has |
| `GET /api/scenarios?network=` | the scenario list with details (kind, node, size, noise, test or training) |
| `GET /api/network?network=` | node positions and pipes for the map |
| `GET /api/analyze/{scenario}` | the verdict, probabilities, ranked nodes, search region, affected nodes, per-node and per-pipe numbers, and the true answer |
| `GET /api/evaluation` | the saved test results |

## 10.2 Dashboard

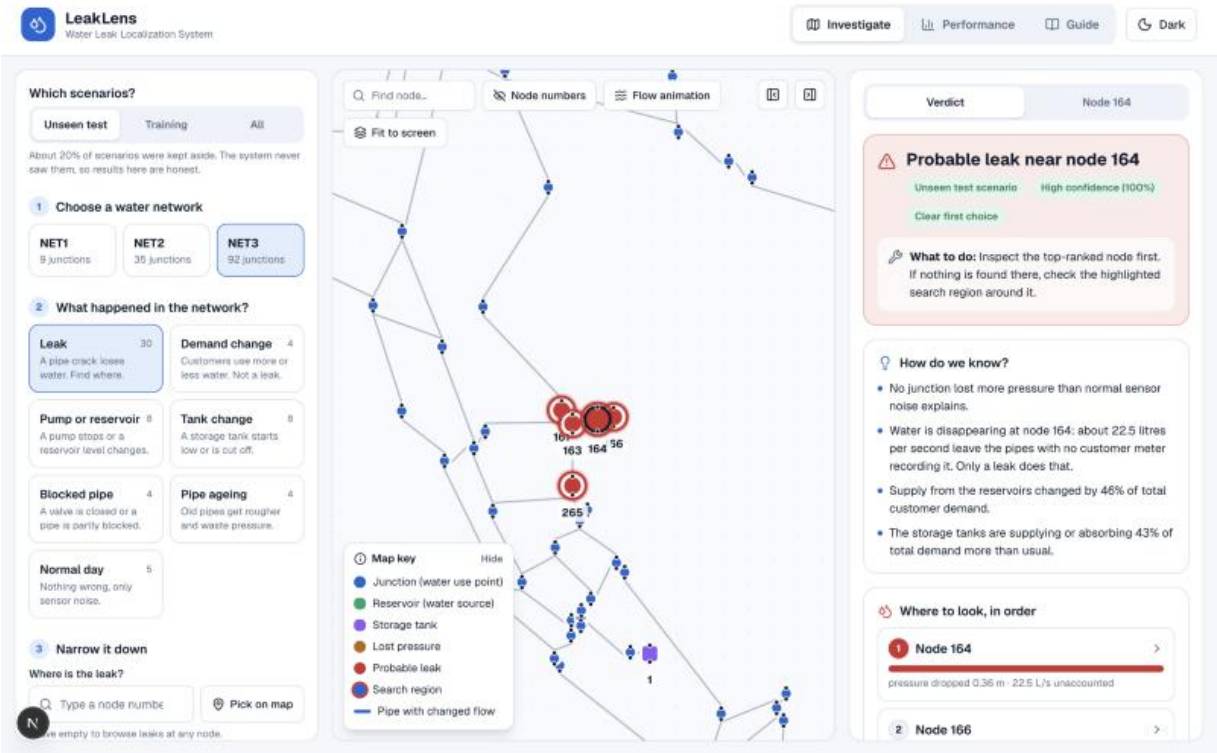



**Figure 6:** Light theme. Left: choose a situation. Centre: the network map. Right: the verdict, the reasons and the nodes to inspect, with an *Unseen test* tag.

The design goal was that a non-specialist can use it without reading a manual:

- **Guided choice, not a giant list.** Step 1 network, step 2 what happened (seven cards), step 3 narrow down: for leaks, type a node number or press *Pick on map*, then choose a size and conditions. A counter shows how many scenarios match; *Surprise me* and next/previous arrows replace scrolling.
- **One-click examples** on the welcome panel.
- **Plain-language verdict:** a headline, a confidence label, a *What to do* line, and reasons written from

the measurements (for example, “22 litres per second leave the pipes with no customer meter recording it”). Technical details are folded away.

- **Explained words:** dotted-underline terms show a definition on hover or tap; a Guide tab lists them.
- **Honest testing built in:** the Unseen test / Training tag on every scenario and a warning if you analyse a training scenario.
- **Light and dark themes,** keyboard-friendly controls, a phone layout, and collapsible side panels.
- **Performance tab:** the results of this report in plain language.

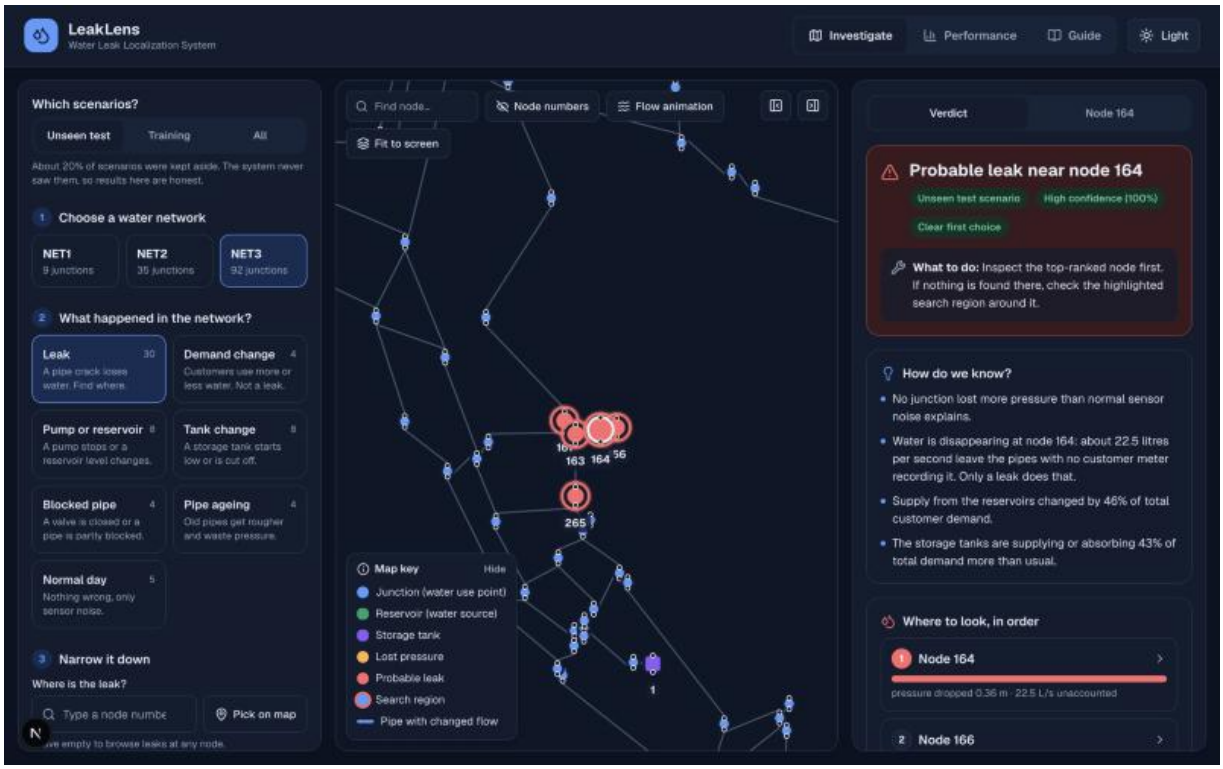



**Figure 7:** Dark theme, a leak found at the first node of the list, with its search region around it.

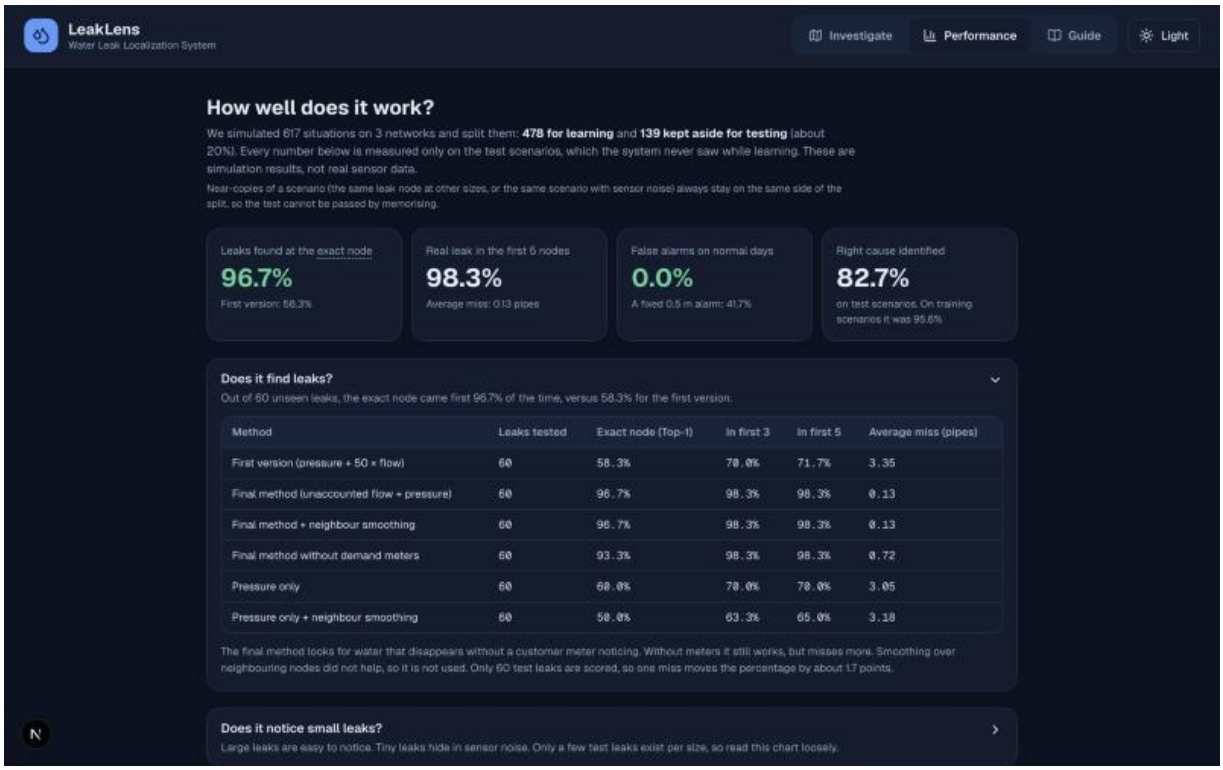



**Figure 8:** The Performance tab: test results with the learning/testing split stated at the top.

### 10.3 What the screen says, and where it comes from

The sentences under "How do we know?" are not written by a language model. They are fixed templates that appear when one of the 14 numbers passes a limit, with the real values filled in. For example, "a pump lost 100% of its flow" appears when the pump-flow number is above 0.5; "water is disappearing at node X" appears when the unaccounted flow is above the noise level.

## 11 How to run it

1. `python simulation/build_scenarios.py` builds the scenario list.
2. `python simulation/generate_dataset.py --force` runs the simulations (about 30 seconds).
3. `python ml_pipeline/train.py` trains the cause model on the training scenarios.
4. `python ml_pipeline/evaluate.py` measures everything on the test scenarios.
5. `cd backend && uvicorn main:app --port 8000` starts the server.
6. `cd frontend && npm run dev` starts the dashboard at `http://localhost:3000`.

## 12 Honest limitations

These matter when you present or rely on the results.

- **Simulation only.** Real networks have real noise, errors in the model and surprises we did not simulate.
- **Small test set.** 139 test scenarios, 60 scored leaks and 12 normal days. One miss moves a percentage by about 1.7 points, and the split was made once.

- **Settings chosen before the split.** The ranking weight and the cut-offs used all scenarios and were not re-tuned.
- **Demand meters are assumed.** Real utilities usually meter an area, not each node. Without per-node meters leak ranking and cause accuracy drop.
- **Small leaks are missed** when everything stays within the sensor noise.
- **The biggest leaks are hard to pinpoint** because they lower pressure everywhere.
- **Causes do not transfer well to unseen networks.** Pump, tank, blockage and ageing events are confused, most in the very small Net1.
- **Gap between training and test accuracy** (95.6% versus 82.7%): the cause model partly memorises.
- **Excluded readings:** two Net3 nodes next to a pump outlet and one hour of pump switching were ignored because they swing wildly for tiny changes. Pump speed changes were not simulated (the simulator does not support them), and some blockage cases did not converge and were dropped.

## 13 What comes next (Semester 2)

Missing and sparse sensors (using graph methods to choose where to put sensors [4]); estimating demand where nodal meters do not exist; several leaks at once; cause classification that carries over to new networks, including graph or algorithm-informed models [5] compared against this interpretable baseline; a larger, repeatedly re-split test; and validation on public benchmark data or real utility data.

## 14 Glossary

| Term | Meaning |
|---|---|
| **Baseline / normal day** | The simulated day with nothing wrong, used as the reference. |
| **Junction, reservoir, tank, pump** | Water use point; water source; storage; machine that adds pressure. |
| **Pressure (m)** | How hard water pushes in a pipe, as metres of water column. |
| **Flow (L/s)** | Litres of water passing a point each second. |
| **Demand** | Water customers actually draw at a node. |
| **Demand meter** | A meter reporting customer use at a node. |
| **Unaccounted flow** | Water leaving a node that no meter records: the leak flow. |
| **Sensor noise** | Small random errors in readings. |
| **Scenario** | One simulated 24-hour situation with a known true answer. |
| **Training / test scenarios** | Scenarios the system learns from / keeps aside to be tested on. |
| **Random forest** | Many small decision trees that vote on the answer. |
| **Precision, recall, F1** | How often an alarm is real; how many real events are caught; a combined score. |
| **False-alarm rate** | Share of normal days that trigger an alarm. |
| **Confusion matrix** | Table of true kind versus the system's answer. |
| **Top-1 / Top-5** | The true leak is first / within the first five nodes. |
| **Pipe hop** | One pipe between two nodes; used to count how far a guess is from the truth. |
| **Search region** | The predicted node and its direct neighbours. |
| **Leave-one-network-out** | Training on two networks and testing on the third. |

## References

[1] A. L. Sousa, A. F. Brochado, E. Rocha, and A. Andrade-Campos, "A state-of-the-art review of scientific, industrial, and commercial developments in water leakage management for water supply systems," *Water Research*, vol. 299, 125853, 2026.

[2] A. Soldevila, G. Boracchi, M. Roveri, S. Tornil-Sin, and V. Puig, “Leak detection and localization in water distribution networks by combining expert knowledge and data-driven models,” *Neural Comput. Appl.*, vol. 34, no. 6, pp. 4759–4779, 2022.

[3] C. Sun, B. Parellada, V. Puig, and G. Cembrano, “Leak localization in water distribution networks using pressure and data-driven classifier approach,” *Water*, vol. 12, no. 1, 54, 2020.

[4] D. U. Leonzio, T. Cattai, P. Bestagini, S. Colonnese, and S. Tubaro, “Centrality-guided node selection for efficient water leak detection,” *IEEE Access*, vol. 14, pp. 24432–24446, 2026.

[5] Z. Zhang and O. Fink, “Algorithm-informed graph neural networks for leakage detection and localization in water distribution networks,” *Reliab. Eng. Syst. Saf.*, vol. 265, 111494, 2026.